Competition-Grade Line Following Robot (LFR) - ESP32-S3

This document serves as the master guide for the LFR project, merging rules, component lists, architectures, algorithms, and code.

---

1 Rulebook Constraints & Track Characteristics

Robot Constraints

- **Max Dimensions**: 25cm(L) × 25cm(W) × 15cm(H)
- **Max Weight**: 1kg (16V DC max voltage restriction)
- **Rules**: Fully autonomous, onboard power, physical kill switch required to cut main power.

Arena & Lines

- **Area**: 15ft x 10ft.
- **Line**: 30mm width, Black on white (or white on black).
- **Intersections**: Min distance between nodes is 25cm, and curve nodes/angles are at least 45 degrees.

Tasks (Trash Collection - Round 2)

- **Payload**: 5x5x5 cm cubes (Max 15g).
- **Rule**: Stored *internally* inside/top of the robot (no dragging). Must pick up from 5-10 spots and dump entirely in a 30x30x5cm target zone.

---

2 Hardware List & Purchasing

Currently Owned

1. ESP32-S3 N16R8 Data Board

2. IBT2 Motor Drivers (x2)
3. 25GA 500 RPM DC Geared Motors (x2)
4. Analog 14 IR Array (Curved/Boomerang)
5. 16-Channel Analog Multiplexer (e.g., CD74HC4067)
6. 3.7V 1S 1500mAh LiPo Batteries (x3 to make 11.1V pack)
7. 1.3" I2C OLED Display (SSH1106) & Rotary Encoder
8. Buzzer, MG90 Servos, HC-SR04+ (3.3V/5V compatible) Ultrasonic Sensor

Need To Buy (Shopping List)

1. **Power**: DC-DC 3A Buck Step-down Power Supply Module (12V to 3.3V Fixed Output), Filtering Capacitors (2200μF, 1000μF).
2. **Mechanics**: 65mm Rubber Wheels x2, Ball Caster Wheel x1, Assorted standoffs/M3 screws.
3. **Task Additions**: Internal Trash Storage Tray, and an extra Micro Servo (e.g., MG90) for payload lifting and grab/release tasks. (Ensure all servos and sensors purchased can strictly operate on native 3.3V).
4. **Sensors**: MPU6050 6-Axis Gyroscope/Accelerometer module (for wobble detection and PID Auto-Tuning).
5. **Misc**: Voltage Detection Sensor Module (25V).

---

3 Wiring & Architecture

> **Note:** For a full, detailed breakdown of why these specific pins were chosen, please consult the independent `ESP32_S3_Pinout_Rules.md` document. It outlines the avoidance of all Boot, Memory, and USB strapping pins. Additionally, while GPIO 10–13 (Motors) fall within the ADC2 range, they are utilized strictly for digital PWM output. This guarantees no hardware conflict when Core 0 activates Wi-Fi, leaving ADC1 safely available for critical analog sensors.

ESP32-S3 Optimized Pinouts

Battery Voltage Measurement

To safely monitor the 11.1V battery on a 3.3V-limited ESP32-S3 logic pin (GPIO 5):

- **Module**: Use a standard Voltage Detection Sensor Module (which features a built-in 5:1 resistor voltage divider: 30kΩ and 7.5kΩ).

Component	Pins (GPIO)	Function
MUX Data	4	ADC1_CH3. Reads the analog value from the MUX.
Battery Volts	5	ADC1_CH4. Reads analog signal from Voltage Detection Module.
I2C Bus (OLED & MPU6050)	8 (SDA), 9 (SCL)	Shared I2C bus for the OLED display and MPU6050 Gyro/Accel. Safe standard pins.
Motors (L/R)	10, 11, 12, 13	IBT2 RPWM/LPWM Controls.
MUX Selects	14, 15, 16, 17	Digital pins (S0-S3) to select IR sensor routing.
Rotary Enc.	6 (CLK), 7 (DT), 18 (SW)	UI input, Start, and Kill Switch.
HC-SR04+	21 (Trig), 48 (Echo)	Obstacle detection. Safe digital I/O.
Servos	1 (Lift), 2 (Grab/Rel.)	Actuators for payload picking & lifting.
Interface	47 (Buzzer)	Buzzer output. Safe digital output.

- **Connection**: Connect **VCC/GND** of the module to the 11.1V battery. Connect **S (Signal)** to **GPIO 5** on the ESP32-S3, and **- (Minus)** to the ESP32 GND.
- **Math**: The module scales the 12.6V fully charged battery down by a factor of 5 (to max 2.52V), remaining completely safe for the 3.3V ADC. Compute voltage in code:

```
Voltage = (analogRead(5) / 4095.0) * 3.3V * 5.0;
```

Power Distribution Schematic & Circuit Arrangement

- **11.1V Main Bus**: The 3S LiPo battery connects directly to the VCC / GND inputs of the two IBT2 motor drivers.
  - **Capacitor Placement**: Place the **2200μF capacitor** in parallel across this 11.1V main bus (between Battery+ and Battery-) close to the motor drivers to absorb high-current voltage spikes.
- **3.3V Step-Down Converter (3A)**: Connected to the 11.1V bus, it drops the voltage to a stable 3.3V. This singular powerful buck converter powers the entirety of the 3.3V ecosystem: the ESP32-S3 (via 3.3V pin, NOT VIN), MG90 Servos (Lift and Grab/Release), HC-SR04+ Ultrasonic, IR Array, MUX board, OLED display, and all pull-up resistors.
  - **Capacitor Placement**: Place the **1000μF capacitor** in parallel across the 3.3V output rails of the step-down converter to prevent logic brownouts when the servos actuate.

(Note: Because the entire logic system runs uniformly on 3.3V, no logic level shifting or voltage dividers are required on any sensor data lines).

---

4 Systems & Algorithms

A. Line Detection

Due to the swept-back shape of the array, the outer edge sensors read lines slightly *later* than forward center sensors.

- **'X' / 'T' Junctions**: Code uses a short buffer (e.g., 20-50ms). When center hits black, and outer hits black shortly after, register intersection.
- **'Y' Splits**: Triggers immediately on extreme outer left/right sensors. The center stays white. Use priority rule to pick a fork.
- **Broken Lines (Gaps)**: If all sensors see white natively in a straight segment, trigger a **coast** state matching previous PWM.
- **Dead Ends**: Rapid 180-degree U-Turn to reset the center sensor lock.
- **End Box Detection**: If all 14 sensors continuously read black (or the target track color) without reverting, this identifies the 30x30cm End Box. The robot immediately halts to secure the +30 points.

B. PID Control

Position error utilizes a "weighted average" summation of all 14 analog streams:

```
Pos = \Sigma(sensor_val[i] \times pos_wt[i]) / \Sigma(sensor_val[i])
Error = Pos - Zero_Center
MotorSpeed = Base_Speed \pm ((Kp * Error) + (Ki * Integral) + (Kd * Deriv))
```

PID Modes:

- **Manual Mode**: Uses hardcoded/user-entered `Kp`, `Ki`, and `Kd` values.
- **Auto Mode**: Uses `kp_auto`, `ki_auto`, and `kd_auto` values, dynamically generated via MPU6050 analysis in the `Debug` testing session.

C. Auto-Tuning Mathematics (Åström-Hägglund Relay & Ziegler-Nichols)

The active "Debug" mode employs the **Relay Feedback Auto-Tuning** algorithm (Åström & Hägglund, 1984) combined with **Ziegler-Nichols closed-loop tuning rules**. The MPU6050 acts as the measurement instrument to dynamically calculate the ultimate gain (K_u) and ultimate period (T_u) of chassis wobble.

1. **Relay Feedback Oscillation**: During a straight line segment, the robot temporarily suspends continuous proportional steering. It applies a fixed "Bang-Bang" steering amplitude (relay amplitude, d) to force a controlled, consistent wobble across the track line.

2. **Wobble Quantification**: The MPU6050's gyroscope (measuring yaw rate ω_z) tracks this forced oscillation. The micro-controller measures:
   - **Peak Amplitude (a)**: The physical amplitude of the resulting angular velocity wobble.
   - **Ultimate Period (T_u)**: The time (in seconds) between two consecutive oscillation peaks.
3. **Ultimate Gain (K_u) Calculation**: Using the Describing Function analysis of a relay, the system's ultimate gain is calculated as:

$$K_u = \frac{4d}{\pi a}$$

   (Where d is the steering relay magnitude, and a is the measured wobble amplitude).
4. **PID Parameter Derivation (Z-N Rules)**: Once K_u and T_u are acquired from the MPU6050 data stream over several oscillation cycles, the mathematically optimized "No-Wobble" PID tuning parameters are established:
   - **Proportional (K_p)**: $K_p = 0.6 \times K_u$
   - **Integral (K_i)**: $K_i = 1.2 \times \frac{K_u}{T_u}$
   - **Derivative (K_d)**: $K_d = 0.075 \times K_u \times T_u$
5. **Boundary Clamping & Application**: These mathematically derived parameters are verified against hard constraints to prevent integer overflow. Once bounded, they update the `kp_auto`, `ki_auto`, and `kd_auto` global variables respectively. Terminating the session guarantees they are saved to EEPROM for the Speed Run phase.

D. Maze Solving (LSRB Pathing)

- **Primary Rule**: Left-Hand Method (Priority: 1=Left, 2=Straight, 3=Right, 4=Back).
- **Exploration Run**: Robot records the path string at each intersection (e.g., `L-S-R-B-L`).
- **Optimization**: After finding the end, compresses using U-Turn ('B') substitution (e.g., `L + B + L = S` and `L + B + R = B`).
- **Speed Run**: During the final measured run, the bot follows the optimized instruction array.

E. Display UI, Telemetry, & EEPROM Memory

All calibration, tuning, and diagnostics are structurally managed through the OLED display and rotary encoder.

1. **Run Mode**:
   - **Run Sub-menu**: Provides explicit selection between **Speed Run** (classic track loop), **Maze Explorer** (solves an unknown maze using LSRB and maps the nodes), and **Maze Runner** (follows the strictly optimized compiled path).

- **Sparklines / Micro-charts**: The bottom 16 pixels display a scrolling line chart of the PID Error, giving a fast visual indicator of oscillation.
- **Acceleration Gauge**: A visual progress bar indicates dynamically scheduled K_p or base PWM scale during S-Curve/Trapezoidal profiling.

2. **Debug (Auto-Tune) Mode**:
   - Runs auto-tuning utilizing the MPU6050.
   - **Web-Based Telemetry Dashboard**: Hosted natively on the ESP32 (via Core 0 & WebSockets) exclusively in Debug mode. Serves an HTML/JS page (via SPIFFS) to plot live PID Errors, Battery Voltage, and Gyro dynamics directly to a laptop/phone using Chart.js.
3. **Calibrate Mode**: Triggers Min/Max boundary sweep logic for array sensors. Thresholds are calculated and **saved automatically and permanently to EEPROM**. The robot will remember its last calibration setup indefinitely and load it instantly on boot, until a new calibration sequence is strictly triggered by the user.
4. **Setup Mode**:
   - **PID Modes**: Toggle between Auto PID, Manual PID, and **Dynamic PID**.
   - **Dynamic Calibration**: Toggle real-time adaptive thresholding on/off.
   - **Default Turn Priority**: Configure the default dead-end / split-path behavior for Speed Runs or Explorer rules ("L/R/S").
   - **Servo Calibration**: Features explicit menu options to set and store the neutral, min, and max positions of the payload Lift and Grab servos.
   - **Network / Modems**: Manually enable/disable Wi-Fi and Bluetooth. *(Note: Any parameter or function toggle located in multiple menus acts uniformly. Toggling Wi-Fi here completely syncs the robot state universally, applying equally to the Debug Web-Dashboard and blocking constraints).*
   - Use Rotary Encoder to edit `Kp`, `Ki`, `Kd`, Component Pinouts, and Motor Speeds.
5. **View Mode**:
   - **Graphical Sensor Array**: Instead of basic numbers, analog reads are visualized as a **live 14-bar histogram** using the SSD1106 graphics buffer, verifying the sensor "curve" instantly.
   - **Path Replay Visualization**: Log and instantly replay the completed string-based maze path on the OLED or the WebSockets dashboard.
6. **EEPROM Storage**: All constants, default turn priorities, and calibration arrays auto-save to EEPROM permanently across power cycles.

F. Advanced Motion Control & Sensor Fusion

To push beyond standard line-following limits, the architecture incorporates real-time dynamics control:

- **Sensor Fusion (Gyro + IR)**: Actively utilizes the MPU6050 yaw (ω_z) data during standard runs. If the robot encounters a significant line gap where all IR sensors read the background, the gyroscope enables closed-loop dead-reckoning to confidently maintain the precise heading until the line is reacquired.

- **Turn Angle Detection via MPU6050**: Validates mathematically true 90° and 180° U-Turns through raw yaw integration. This completely replaces standard time/delay-based turn calculations, dramatically increasing accuracy.

- **Trapezoidal / S-Curve Motion Profiling**: Implements algorithmic acceleration and deceleration curves rather than instantaneous PID base speed jumps. Ramping the motor PWMs prevents traction loss (wheel slip) and chassis jolts when accelerating out of corners or hard braking into acute intersections.

- **PID Gain Scheduling**: Replaces static PID tuning with dynamic arrays. The system smoothly interpolates between different K_p, K_i, and K_d sets depending on the track curvature and forward velocity. High-speed straightaways utilize high K_d and low K_p for stability, while sharp, slow corners seamlessly shift to a high K_p for aggressive turning.

G. Sonar Sensor (HC-SR04+) Implementation

Crucial for the NRF26 rulebook trash collection mechanics:

- **Non-Blocking FreeRTOS on Core 0**: The standard `pulseIn()` function inherently blocks Core 1's PID controller loop. To maintain flight-controller speeds, Sonar runs entirely asynchronously via a hardware interrupt + timer architecture mapped to Core 0.

- **Detection Logic**: Safely identifies general obstacles, triggers the payload grab sequence when a target cube enters the 3–5cm physical striking boundary, and validates physical proximity against the 30x30x5cm end target zone wall.

- **Payload Carry Override**: Once a payload is actively grabbed and lifted into the internal storage mechanism, the sonar signal reading is software-muted and purposely neglected. This strict condition prevents false-positive obstacle halts caused by the payload sitting within the sensor's physical beam during transport.

H. Payload Handling & Trash Dump Logic

The complete system for the Round 2 trash payload task:

- **Tracking (Multi-Payload Memory)**: Uses a payload counter and a dynamically allocated visited-node memory array to track whether a trash cube has been gathered from the 5–10 possible pick-up spots.

- **Pickup Algorithm**: Standard PID logic is preempted; the robot halts precisely via Sonar + IR, deploys the grabber, lifts the payload smoothly into internal chassis storage via GPIO 1 & 2, and restarts forward acceleration profiling.

- **Dump Zone Arrival Algorithm**: Leverages a dual-sensor condition logic. IR verifies the bold black target line boundary, whilst the Sonar verifies distance to the drop-off gate. Servos autonomously reverse to empty storage and finish the course.
- **Inspection & Retraction Sequence**: To comply with the strict starting box dimensions (25x25x15cm), the robotic gripper immediately retracts to a compact tucked position during system boot, run termination, or upon activation of the kill-switch.

---

5 Assembly & Best Practices

1. **Isolation & Power**: Even though the external 3.3V buck converter can supply 3 Amps, ensure the servos are powered directly from the buck converter's output pads, *not* daisy-chained through the ESP32-S3 development board. This prevents servo stall-currents from browning out the microcontroller.
2. **IR Array Height**: The 14 IR Array should sit around **3mm - 5mm** off the floor.
3. **PID Tuning Flow**:
   - Set Kp, Ki, Kd to 0.
   - Ramp `Kp` up until the chassis visibly oscillates/wiggles over the line.
   - Reduce `Kp` slightly, then ramp `Kd` to smooth/dampen those wobbles out.
   - Increase `Ki` slightly only if the robot drifts heavily on long wide curves.

---

6 Advanced ESP32-S3 Features: Dual-Core & PSRAM

FreeRTOS Dual-Core Utilization

The ESP32-S3 features two Xtensa LX7 cores. To prevent operations like updating the OLED screen from interrupting the fast motor control loop, tasks are split using FreeRTOS:

- **Core 0 (PRO_CPU)**: Handles non-time-critical background tasks such as I2C OLED rendering, rotary encoder UI processing, and basic state tracking.
- **Core 1 (APP_CPU)**: Dedicated exclusively to high-speed robotic operations. Executes MUX analog reads, PID error calculations, and motor PWM updates, guaranteeing zero-latency line following.
- **Concurrency & Protection**: To prevent hardware collisions (e.g., I2C bus and timing crashes), peripheral access is strictly isolated. **Only Core 0** is allowed to touch the I2C bus. Core 1 passes shared data safely using **FreeRTOS Queues** or **Mutexes**.
- **Implementation**: Enabled via `xTaskCreatePinnedToCore()` during standard setup, enforcing strict parallelism.

Octal PSRAM (8MB) Utilization

With 8MB of PSRAM, bulk data is shifted off the limited internal SRAM to prevent stack-overflows:

- **Maze Path Arrays**: Node histories constraints are completely lifted. Intersections are dynamically allocated in PSRAM using `ps_malloc()`.
- **Telemetry Data**: Intensive run logs or full graphical display buffers can safely occupy PSRAM.
- **Pin Safety**: Because our hardware diagram strictly avoids GPIO 26–38, the Octal SPI lines to the PSRAM suffer zero hardware contention.

Over-The-Air (OTA) Updates & Wireless Compliance

Since the robot is tightly packed for competition and already uses Wi-Fi for telemetry, **ArduinoOTA** (or ESP-IDF OTA) is implemented. This allows for rapid code flashing completely wire-free, without disconnecting the main battery or exposing USB ports during track testing.

Crucial Rulebook Compliance: To strictly adhere to the "No Wireless Communication" rule, the ESP32-S3 Wi-Fi and Bluetooth modems are **disabled by default** on boot. Wireless telemetry and OTA are exclusively enabled via manual activation inside the **Setup Mode** running off the OLED Menu. In "Run Mode", the network is guaranteed completely off to pass technical inspection seamlessly. Any feature (like Wi-Fi) toggled across different menus will mathematically sync globally to prevent accidental broadcasts.

---

7 References & Links

- **HC-SR04+ Ultrasonic Sensor**: RoboticsBD - 3.3V/5V Compatible Sonar Sensor
- **Voltage Detection Module**: RoboticsBD - 25V Voltage Sensor Module
- **PID Auto-Tuning Research**: Åström, K. J., & Hägglund, T. (1984). "Automatic tuning of simple regulators with specifications on phase and amplitude margins." *Automatica*, 20(5), 645-651. Wikipedia: Ziegler-Nichols tuning